

Diarizer

Pietro De Angeli

16/9/2024

Contents

1	Requirements	3
2	Installation	4
3	Models	4
3.1	ASR - SST Multilingual	4
3.2	VAD - Marblenet	5
3.3	SpeakerID - TitaNet-L	5
4	Code Structure	6
4.1	Overview	6
4.2	AudioSample	6
4.3	Diar_application	6
4.3.1	ASR_Diar.py	6
4.3.2	Constructor	7
4.3.3	add_audiofile()	7
4.3.4	set_config()	8
4.3.5	transcript_infer()	8
4.3.6	diarization()	9
4.3.7	convertJSON()	10
4.3.8	conversion.py	10
4.3.9	resample.py	11
4.4	Models	12
5	Performance	13
5.1	Memory	13
5.1.1	Initialization	13
5.1.2	Adding a file	13

5.1.3	Transcription	14
5.1.4	Diarization	14
5.1.5	All	15
5.2	Execution Time	15
6	How to Run the Program	15
7	NVIDIA Library Modifications	16
7.1	rnnt_decoding.py	16
7.2	diarization_utils.py	16
7.3	clustering_diarizer.py	16
8	Future Changes	16
9	Support	17

This document describes the functionality of the diarizer built using the ASR models provided by NVIDIA.

The program takes as input audio files up to 5 minutes long (preferably 16KHz and mono-channel) and outputs a JSON file containing the transcription, a speaker identifier, and word timestamps.

This document briefly describes the following topics:

- Requirements
- Installation
- Models
- Code Structure
- Performance
- How to Run the Program
- NVIDIA Library Modifications
- Future Changes
- Support

1 Requirements

1. Windows 10 (or later)
2. Docker
3. 30 GB of disk space
4. 10 GB of RAM
5. NVIDIA GPU (optional)

The NVIDIA GPU is not mandatory but is useful to reduce inference times.

2 Installation

Assuming Docker is already installed (Requirement N1), proceed with downloading the image.

1. Install the image: `docker pull <path_DockerHub>/nvidia_asr_container_1`
2. Create the Docker container: `docker create --name <container_name> <path_DockerHub>/nvidia_asr_container_1 --shm-size=100mb`

If you have an NVIDIA GPU, also use the flag `--gpus all`

The Shared memory flag is used to solve issues when processing audio longer than one minute, as Docker's default shared memory (64MB) is insufficient.

3 Models

3.1 ASR - SST Multilingual

https://catalog.ngc.nvidia.com/orgs/nvidia/teams/nemo/models/stt_multilingual_fastconformer_hybrid_large_pc.

This model is a checkpoint of the ASR model pre-trained by NVIDIA on the following languages:

- Belarusian (466 hrs)
- German (2509 hrs)
- English (8580 hrs)
- Spanish (1400 hrs)
- French (1799 hrs)
- Croatian (1659 hrs)
- Italian (501 hrs)
- Polish (165 hrs)
- Russian (1692 hrs)
- Ukrainian (151 hrs)

This model is used to transcribe audio and also provides word-level timestamps.

Fine-tuning is a valid option to allow transcription of languages it has not been trained on.

In the context of diarization, this model is used to obtain transcriptions and their timestamps, which are then used to realign the output from speakerID + VAD with the transcriptions.

Note: The model cannot return the language it is transcribing, so two options are available:

- Use an external model for language detection
- Provide the expected language together with the input audio

3.2 VAD - Marblenet

https://catalog.ngc.nvidia.com/orgs/nvidia/teams/nemo/models/vad_marblenet. This model is used for Voice Activity Detection. Given an input audio file, it returns timestamps where the audio is classified as *Speech*.

In the context of diarization, it is used to extract the speaker for the *Speech* segments. The timestamps are used for both SpeakerID and alignment with the audio transcription.

3.3 SpeakerID - TitaNet-L

https://catalog.ngc.nvidia.com/orgs/nvidia/teams/nemo/models/titanet_large. This model is used for speaker identification. Given an input audio segment, it returns the embedding of the speaker. If there are multiple speakers, it returns the one with the most vocal activity.

In the diarization process, audio chunks with detected speech activity are used to extract the speaker's embedding.

4 Code Structure

4.1 Overview

The ASR-related code is located in `/workspace/tirocinio/Diar_application`, which contains the following directories: *AudioSample* (with example audio files), *models* (where the models are stored), and *tutorials and test*.

The main program directory is called *Diar_application*.

4.2 AudioSample

This folder contains example audio files used for testing. You can delete them and manually add your own files to be diarized (the application does not support file uploads).

4.3 Diar_application

This directory contains the core logic of the internship project. It includes a *data* subdirectory that stores intermediate and final files created during diarization. The final outputs are located in `./output`.

To run the program, execute the *interactive_app.py* file whose `main()` function starts the main application. The *InteractiveApp* class has a constructor that creates an instance of the *Diarizer* class. The *run()* function displays a menu with the following options:

- `add`: select a new audio file to diarize (provide path as input).
- `tra`: transcribe the first audio file in the queue.
- `dia`: run VAD + SpeakerID and print the results.
- `all`: combine the previous two functions and write the formatted result to a JSON file.
- `exit`: exit the program.

4.3.1 ASR_Diar.py

This file defines the *Diarizer* class and its functions.

4.3.2 Constructor

Loads and sets the configurations for the ASR and diarization models. The VAD and SpeakerID models are loaded later (at diarization time). All models are loaded locally from the container, not downloaded from the internet.

4.3.3 add_audiofile()

Adds an audio file for processing and updates the class properties.

Parameters:

- audio_filepath: Path to the audio file.

Class attributes updated:

- self.AUDIO_FILEPATH: Path to the audio file.
- self.AUDIO_FILENAME: Filename without extension.
- self.duration: Duration of the audio file in seconds.

Actions performed:

1. Loads the audio file and calculates its duration.
2. If the audio is not sampled at 16kHz, performs resampling and updates the file path and name.

```
90 def add_audiofile(self, audio_filepath):
91     # Initialize directories and filenames
92     self.AUDIO_FILEPATH = audio_filepath
93     self.AUDIO_FILENAME = os.path.splitext(os.path.basename(self.AUDIO_FILEPATH))[0]
94     self.duration = 0
95
96     # Audio manipulation
97     signal, sr = load(self.AUDIO_FILEPATH, sr=None)
98     self.duration = get_duration(y=signal, sr=sr)
99
100    # Resampling if necessary
101    if sr != 16000:
102        self.AUDIO_FILEPATH = resample.resample_audio(self.AUDIO_FILEPATH)
103        self.AUDIO_FILENAME = os.path.splitext(os.path.basename(self.AUDIO_FILEPATH))[0]
```

Note: This function could be improved by implementing a queue to handle multiple files instead of one at a time.

4.3.4 set_config()

Creates a JSON manifest file for diarization input. This method generates an `input_manifest.json` file in the `data` directory containing necessary information for diarization, such as audio path, duration, and metadata. It must be called every time a new file is added.

Used attributes:

- `self.AUDIO_FILEPATH`: sets the path to the audio file.
- `self.duration`: sets the duration of the audio file.

```
120     def set_config(self):
121
122         # Create a manifest file for input
123         meta = {
124             'audio_filepath': self.AUDIO_FILEPATH,
125             'offset': 0,
126             'duration': self.duration,
127             'label': 'infer',
128             'text': '-',
129             'num_speakers': None,
130             'rttm_filepath': None,
131             'uem_filepath' : None
132         }
133         with open(os.path.join(self.data_dir, 'input_manifest.json'), 'w') as fp:
134             json.dump(meta, fp)
135             fp.write('\n')
```

4.3.5 transcript_infer()

Performs transcription of the audio file and returns word-level hypotheses with timestamps.

Returns:

- `word_hyp` (`Dict[str, List[str]]`): dictionary with filename as key and list of transcribed words as value.
- `word_ts_hyp` (`Dict[str, List[List[int]]`): dictionary with filename as key and list of timestamps (start and end in ms) for each word.

Example:

- `word_hyp = 'audio_filename': ['Hello', 'world']`
- `word_ts_hyp = 'audio_filename': [[0, 500], [501, 1000]]`

```

121 def transcrip_infer(self):
122     # Transcription
123     hypotheses = self.asr_model.transcribe([self.AUDIO_FILEPATH], return_hypotheses=True, batch_size=2)
124
125     if type(hypotheses) == tuple and len(hypotheses) == 2:
126         hypotheses = hypotheses[0]
127
128     # Parsing transcription
129     timestamp_dict = hypotheses[0].timestamp
130     time_stride = 8 * self.asr_model.cfg.preprocessor.window_stride
131
132     word_hyp: Dict[str, List[str]] = {}
133     word_ts_hyp: Dict[str, List[List[int]]] = {}
134
135     word_hyp[self.AUDIO_FILENAME] = list()
136     word_ts_hyp[self.AUDIO_FILENAME] = list()
137
138     for stamp in timestamp_dict['word']:
139         word_hyp[self.AUDIO_FILENAME].append(stamp['char'] if 'char' in stamp else stamp['word'])
140         word_ts_hyp[self.AUDIO_FILENAME].append([stamp['start_offset'] * time_stride, stamp['end_offset'] * time_stride])

```

4.3.6 diarization()

Performs diarization on an audio file and aligns the results with the transcription.

Steps performed:

1. Calls `self.asr_diar_offline.run_diarization` to process diarization.
2. Merges diarization results with transcription and timestamps using `self.asr_diar_offline.get_transcript_with_speaker_labels`.
3. Releases used memory to avoid leaks.
4. Reads and prints the final speaker-labeled transcription from file.

Output file:

- JSON file with speaker labels saved in the `pred_rttms` directory.

```

144 def diarization(self, word_hyp, word_ts_hyp):
145     diar_hyp, _ = self.asr_diar_offline.run_diarization(self.cfg, None)
146
147     self.asr_diar_offline.get_transcript_with_speaker_labels(diar_hyp, word_hyp, word_ts_hyp)
148
149     del diar_hyp
150     #del asr_diar_offline
151     gc.collect()
152
153     # Print results
154     def read_file(path_to_file):
155         with open(path_to_file) as f:
156             contents = f.read().splitlines()
157             return contents
158
159     transcription_path_to_file = f"{self.data_dir}/pred_rttms/{self.AUDIO_FILENAME}.json"
160     transcript = read_file(transcription_path_to_file)
161
162     for line in transcript:
163         print(line)

```

4.3.7 convertJSON()

Converts the diarization result JSON and cleans up intermediate files.

Steps:

1. Converts the JSON result from `pred_rttms` into a standard JSON format in `output`.
2. Deletes intermediate files related to the specific audio file.

```
165 def convertJSON(self):
166     input_file = f"/workspace/tirocinio/Diar_application/data/pred_rttms/{self.AUDIO_FILENAME}.json"
167     output_file = f"/workspace/tirocinio/Diar_application/data/output/{self.AUDIO_FILENAME}.json"
168     conversion.convert_json_from_file(input_file, output_file)
169
170     # Routine di pulizia dei file intermedi.
171     pattern = f"/workspace/tirocinio/Diar_application/data/pred_rttms/{self.AUDIO_FILENAME}.*"
172     files_to_delete = glob.glob(pattern)
173
174     for file_path in files_to_delete:
175         try:
176             os.remove(file_path)
177         except OSError as e:
178             print(f"Error deleting file {file_path}: {e}")
179
180     patterns = [
181         f"/workspace/tirocinio/Diar_application/data/pred_rttms/{self.AUDIO_FILENAME}.*",
182         f"/workspace/tirocinio/Diar_application/data/pred_rttms/{self.AUDIO_FILENAME}_gecko.*"
183     ]
184
185     for pattern in patterns:
186         files_to_delete = glob.glob(pattern)
187         for file_path in files_to_delete:
188             try:
189                 os.remove(file_path)
190             except OSError as e:
191                 print(f"Error deleting file {file_path}: {e}")
```

4.3.8 conversion.py

Implements two main functions:

1. `convert_json_from_file`: converts a transcription JSON into a structured format.
2. `convert_iso_to_language`: maps ISO 639-1 codes to human-readable language names.

Note: Some parameters such as audio channel info are defaulted to *None* and should be customized by the company.

```

5  def convert_json_from_file(input_file, output_file):
6
7      with open(input_file, 'r', encoding='utf-8') as file:
8          input_data = json.load(file)
9
10         output_json = {
11             "name": None,
12             "country": None,
13             "tokens": [],
14             "channel_name": None,
15             "stream_url": [None],
16             "transcription_content": input_data.get("transcription", ""),
17             "primary_language": None,
18             "media_type": None
19         }
20
21         for word_data in input_data["words"]:
22             number = re.findall(r'\d+', word_data["speaker"])
23             if number:
24                 speaker = f"spk{number[0]}"
25             else:
26                 speaker = "Unknown"
27             token = {
28                 "data": word_data["word"].strip(),
29                 "language": None,
30                 "speaker": {
31                     "id": speaker,
32                     "check": False,
33                     "name": None
34                 },
35                 "start_millis": int(word_data["start_time"] * 1000),
36                 "end_millis": int(word_data["end_time"] * 1000)
37             }
38             output_json["tokens"].append(token)
39
40         with open(output_file, 'w', encoding='utf-8') as outfile:
41             json.dump(output_json, outfile, ensure_ascii=False, indent=4)

```

4.3.9 resample.py

Resamples audio to 16kHz mono, which is required for all NVIDIA models.
Steps:

1. Checks if input file exists. If not, returns 0.
2. Extracts the filename.
3. Creates a `resampled` directory.

4. Loads and converts audio to mono 16kHz.
5. Saves it as a WAV file and returns the new path.

```
8 def resample_audio(input_path):
9     # Verifica se il file esiste
10    if not os.path.isfile(input_path):
11        print(f"Il file {input_path} non esiste.")
12        return 0
13
14    # Estrai il nome del file senza estensione
15    filename = os.path.basename(input_path)
16    name, _ = os.path.splitext(filename)
17
18    # Percorso per il nuovo file
19    output_dir = "/resampled"
20    os.makedirs(output_dir, exist_ok=True)
21    output_path = os.path.join(output_dir, f"{name}.wav")
22
23    # Carica il file audio
24    audio = AudioSegment.from_file(input_path)
25
26    # Converte in mono e imposta il frame rate a 16kHz
27    audio = audio.set_channels(1).set_frame_rate(16000)
28
29    # Esporta il nuovo file audio
30    audio.export(output_path, format="wav")
31
32    return output_path
```

4.4 Models

Stores model checkpoints used by the application. These can be restored or modified via constructor parameters or manifest files. Models are listed in the dedicated section.

5 Performance

This section describes performance in terms of memory and execution time. All data was collected on a machine without an NVIDIA GPU, so inference times are expected to be lower on GPU-enabled systems.

Performance was measured using 5-minute audio files with one or two speakers. The presence of multiple speakers may increase memory usage and diarization inference time.

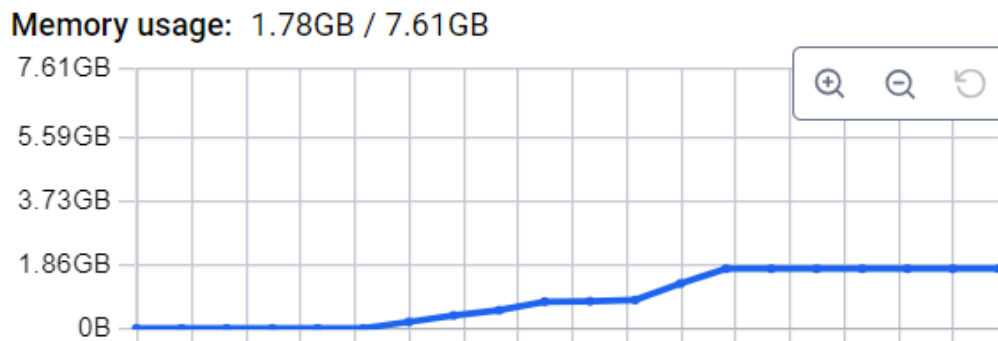
5.1 Memory

This section analyzes memory usage during significant processes of the application.

Data was collected using Docker's container statistics tool, so measurements include operating system overhead.

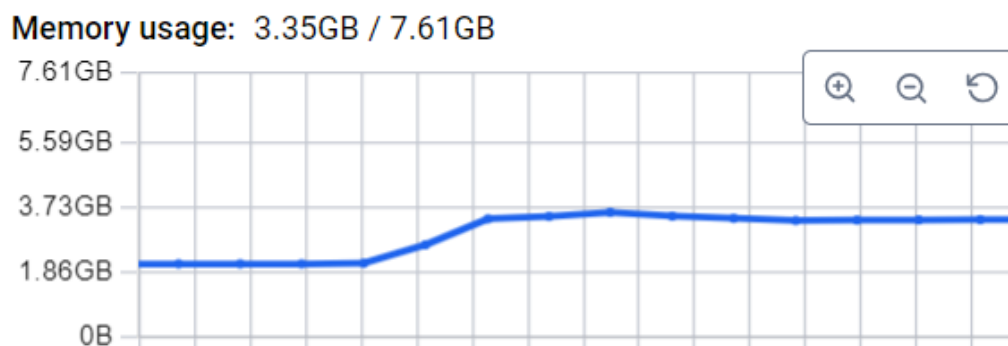
5.1.1 Initialization

Memory usage at model loading (ASR and Clustering Diarizer) is about 2GB.



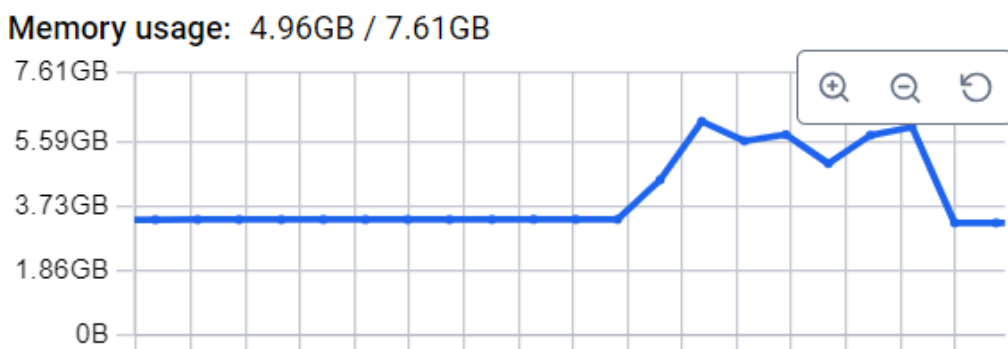
5.1.2 Adding a file

When the first file is added, memory usage increases to about 3.5GB due to the loading of VAD and SpeakerID models.



5.1.3 Transcription

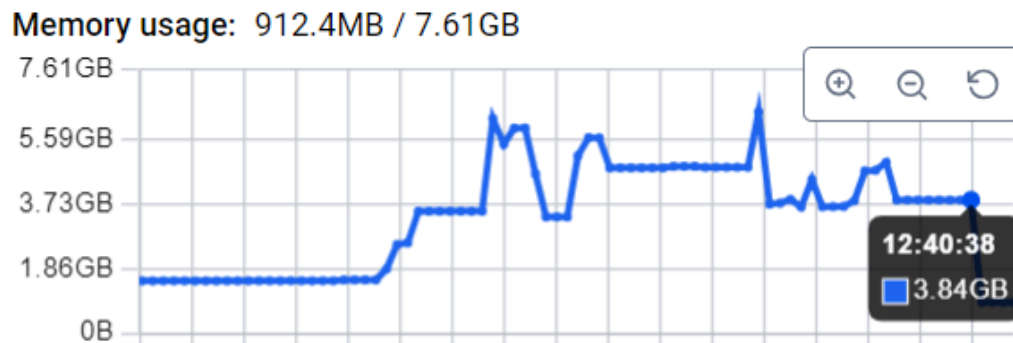
During transcription, memory usage peaks around 6GB.



5.1.5 All

Running the *all* command results in similar memory usage to the previous phases. Two 5-minute audio files were processed in this example.

Note: To optimize RAM usage, it's recommended to use the *all* command which triggers Python's garbage collector.



5.2 Execution Time

Time is measured as (process duration / audio duration).

Initialization takes about 1 minute, due to model loading.

Adding a file is nearly instantaneous.

Transcription time depends on audio length. For shorter clips the factor is about 1/6, increasing to 2/6 for 5-minute audio.

Diarization time is around 1/6 for audio with one or two speakers, and may increase with more speakers.

All command execution time is about 1/3, as it is slightly optimized.

6 How to Run the Program

To run the program, follow these steps:

1. Follow the steps in the Installation section.
2. Start Docker.
3. Start the container with the command: `docker start <container_name>`
`&& docker exec -it <container_name> bash`
This will also open a terminal to interact with the container.
4. Navigate to the *Diar_application* folder using:
`cd /workspace/tirocinio/Diar_application`

5. Start the program with: `python interactive_app.py`

7 NVIDIA Library Modifications

To customize certain features and fix issues, some modifications were made to NVIDIA's libraries.

7.1 rnnt__decoding.py

To make the NVIDIA tutorial for word-level timestamps work, it was necessary to modify the file: `/usr/local/lib/python3.10/dist-packages/nemo/collections/asr/parts/submodules/rnnt_decoding.py` by adding:

```
1 import copy
```

7.2 diarization_utils.py

In the file `/usr/local/lib/python3.10/dist-packages/nemo/collections/asr/parts/Utils/diarization_utils.py`, the `init()` function was modified to accept a Diarizer object and store it. Additionally, the `run_diarization()` function was modified to use the saved Clustering Diarizer model instead of loading it each time it is invoked.

7.3 clustering_diarizer.py

Modifications were made to the file `/usr/local/lib/python3.10/dist-packages/nemo/collections/asr/models/clustering_diarizer.py` as described in: <https://github.com/NVIDIA/NeMo/issues/5637>.

8 Future Changes

Possible future improvements include:

- Expanding transcription language support (fine-tuning the ASR model)
- Multiscale diarization to detect overlapping voices
- Parallel processing (requires more hardware resources)
- File queue handling, so files don't need to be processed one at a time
- Remote file upload functionality

- Providing audio file parameters (currently default to None)
- Recognizing known speakers
- Aligning the same speaker across different audio files (avoid random numbering)

9 Support

For more information, refer to the code documentation, the markdown file available in the container, NVIDIA's official documentation, or contact me at:

`pietro.deangeli02@gmail.com`.